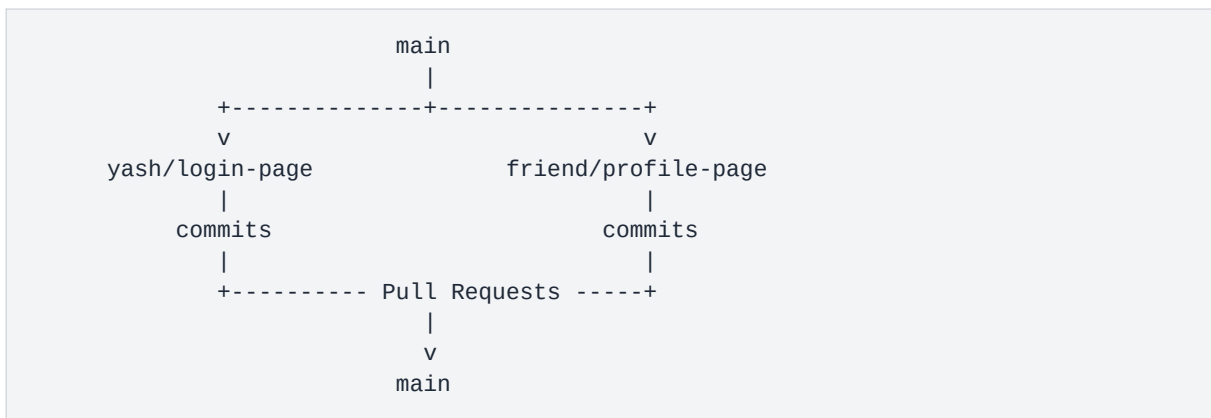


CampusConnect

Git Branching & Pull Request Workflow — for you and your teammate

The golden rule: never push directly to **main**. Every change — yours or your teammate's — goes through a **feature branch** and a **Pull Request (PR)** before it lands in **main**. This keeps **main** stable and working at all times.

The Big Picture



You and your friend each work on your own branch. When a feature is ready, you open a Pull Request asking to merge your branch into main. Someone reviews it, then it gets merged.

Rule for main: keep it stable and working at all times.

■ **Don't push feature work directly to main**

■ **Create a branch → work → push branch → open a PR → merge into main**

STEP 1 Update main before starting new work

Before starting anything new, always make sure your local **main** matches GitHub. This is the single habit that prevents most headaches.

```
git checkout main
git pull origin main
```

STEP 2 Create your feature branch

Say you're building the Dashboard feature:

```
git checkout -b feature/dashboard
```

Check which branch you're on any time with:

```
git branch

* feature/dashboard
main
```

*The * marks your current branch.*

STEP 3 Do your normal coding

Work on your files as usual (client/, server/, etc.). When you're ready to save progress:

```
git status
git add .
git commit -m "Add dashboard page"
```

STEP 4 Push your branch (not main)

```
git push -u origin feature/dashboard
```

Notice you are pushing **feature/dashboard**, never **main**. This sends your local branch up to GitHub as its own branch.

```
local feature/dashboard
    |
    v
GitHub feature/dashboard
```

STEP 5 Open a Pull Request on GitHub

Open the repo on GitHub. It will usually show a banner like:

```
"feature/dashboard had recent pushes -- Compare & pull request"
```

Click **Compare & pull request**, then confirm:

```
base: main  
compare: feature/dashboard
```

This means: “merge my feature/dashboard changes into main.” Click **Create Pull Request**.

STEP 6 Your friend does the exact same thing

Same steps, different branch name — e.g. for the Profile Page:

```
git checkout main  
git pull origin main  
git checkout -b feature/profile  
  
# ... work on the feature ...  
  
git add .  
git commit -m "Add profile page"  
git push -u origin feature/profile
```

Then open a PR: **feature/profile** → **main**.

STEP 7 Review each other's PR before merging

Before merging, look through the changed files in the other person's PR. If everything looks good, click **Merge Pull Request** → **Confirm merge**. Now those changes are part of main.

STEP 8 After someone else merges into main, sync your branch

Say your friend's **feature/profile** just got merged into main, but you're still working on **feature/dashboard**. Your local main is now behind GitHub, so before you continue (or merge your own branch), update it:

```
git checkout main
git pull origin main

git checkout feature/dashboard
git merge main
```

If there are no conflicts, Git will fast-forward or create a merge commit automatically. Then push:

```
git push
```

Your branch now has main's latest changes **plus** your own dashboard work.

STEP 9 Handling a merge conflict

This is the part worth understanding well. Suppose you both edited the same file, **Navbar.tsx** — you changed it, and your friend's version (already merged into main) also changed it.

When you run:

```
git checkout main
git pull origin main

git checkout feature/dashboard
git merge main
```

Git may report:

```
CONFLICT (content): Merge conflict in Navbar.tsx
```

Open the file. You'll see conflict markers like this:

```
<<<<<< HEAD
your changes
=====
friend's changes
>>>>>> main
```

Decide what the final code should look like, then delete the three marker lines (<<<<<<, =====, >>>>>>) entirely. Once the file looks correct:

```
git add .
git commit -m "Resolve merge conflict"
git push
```

Your existing PR updates automatically — no need to open a new one.

Branch Naming Convention

Keep **main** reserved for stable code only. Name every branch after what it actually does:

Type	Prefix	Examples
New feature	feature/	feature/login-page, feature/dashboard, feature/chat
Bug fix	fix/	fix/login-error, fix/cors-error, fix/socket-connection
Documentation	docs/	docs/update-readme

Since CampusConnect has both frontend and backend, avoid branches like **frontend-branch** or **backend-branch**. Instead, branch by **feature** — e.g. **feature/user-profile** can contain changes in both **client/** and **server/** if that one feature needs both. This scales much better as the project grows.

Your Daily Workflow, Summarized

```
# 1. Start new work
git checkout main
git pull origin main
git checkout -b feature/my-feature

# 2. Work on your code, then:
git status
git add .
git commit -m "Add my feature"
git push -u origin feature/my-feature

# 3. On GitHub:
#   Open PR: feature/my-feature -> main
#   Get it reviewed, then merged

# 4. After merge, clean up:
git checkout main
git pull origin main
git branch -d feature/my-feature

# 5. Start the next feature
git checkout -b feature/another-feature
```

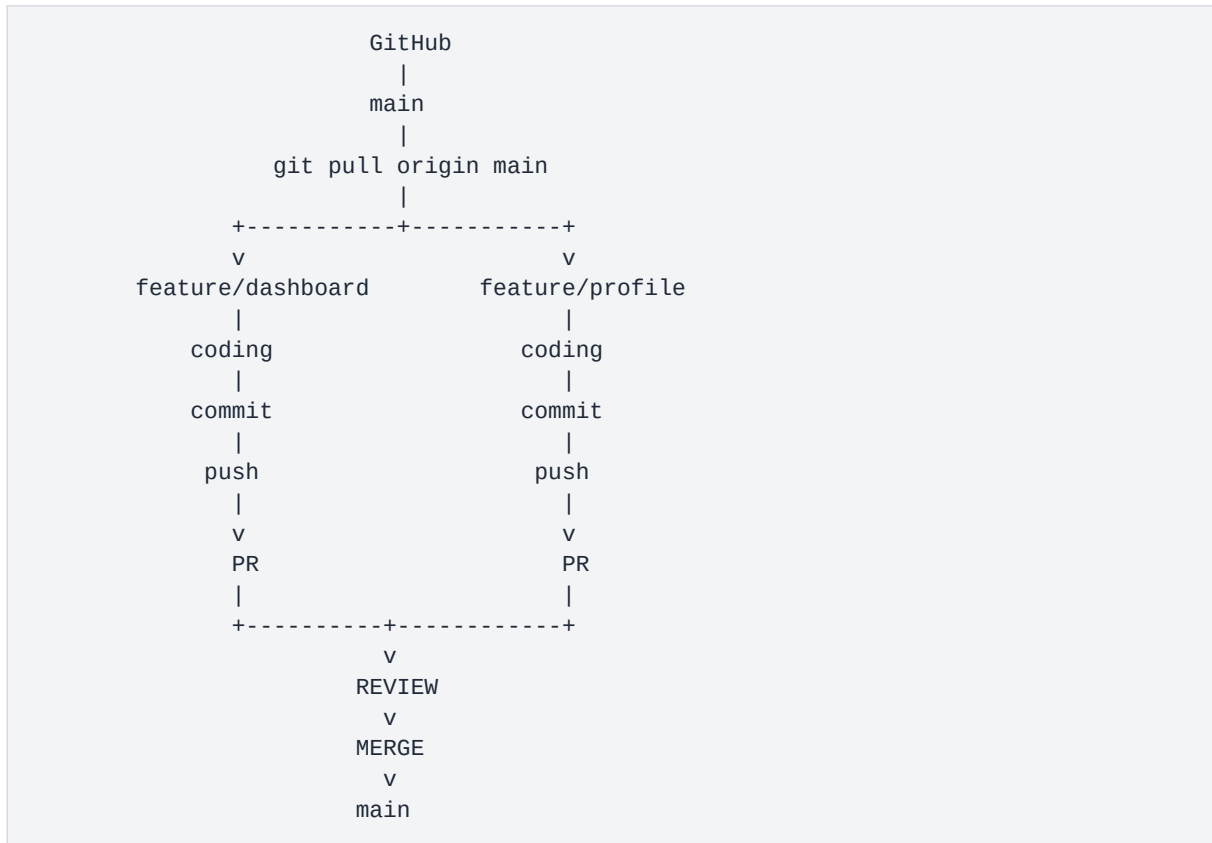
The One Rule That Saves the Most Headaches

Always update main before creating a new branch.

■ **git checkout main → git pull origin main → git checkout -b feature/new-feature**

■ Don't branch off an old feature branch — your new branch may start from outdated code

The Complete CampusConnect Workflow



Bottom line for you two: nobody touches main directly. Every change lives on its own feature/ branch, gets pushed, opened as a PR, reviewed by the other person, and only then merged into main. When conflicts show up, they're solved right in the file with the <<<<<< markers, not by panicking.